Project progress report

Name- Sourabh Khamankar

Project Title: TESLA STOCK PRICE PREDICTION USING DEEP LEARNING

Abstract

Stock price prediction is one of the most challenging problems in financial analytics due to the volatile and dynamic nature of stock markets. Accurate forecasting of stock prices can assist investors, traders, and financial institutions in making informed decisions and reducing investment risks. This project focuses on predicting Tesla Inc. stock closing prices using Deep Learning techniques.

Historical Tesla stock market data was collected and analyzed to understand market behavior and trends. Exploratory Data Analysis (EDA) was performed to identify patterns, trends, distributions, and relationships among stock attributes. After preprocessing and feature engineering, three sequential Deep Learning models—Simple Recurrent Neural Network (SimpleRNN), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU)—were developed and trained.

The performance of all models was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score. Hyperparameter tuning was performed using Random Search to optimize model performance. Among all models, GRU achieved the best predictive performance with the highest R^2 Score and lowest prediction error. The final GRU model was deployed using Streamlit to provide an interactive stock forecasting application capable of predicting Tesla stock prices for the next 1, 5, and 10 days.

---

1. Introduction

The stock market plays a crucial role in the global economy and serves as a platform for investment and wealth generation. However, stock prices are influenced by numerous factors such as company performance, market sentiment, economic conditions, geopolitical events, and investor behavior. Due to these influences, predicting stock prices accurately remains a complex task.

Traditional statistical forecasting techniques often struggle to capture long-term temporal dependencies present in stock market data. Deep Learning models, especially Recurrent Neural Networks (RNNs), have demonstrated strong performance in sequence prediction problems. Models such as LSTM and GRU are specifically designed to retain historical information and learn temporal relationships over extended periods.

The primary objective of this project is to develop and compare multiple Deep Learning architectures for Tesla stock price prediction and identify the most suitable model for deployment.

---

2. Problem Statement

The objective of this project is to predict Tesla stock closing prices using historical stock market data. The project involves developing multiple Deep Learning models and comparing their performance.

The specific objectives are:

- Analyze Tesla historical stock data.
- Perform data preprocessing and feature engineering.
- Create sequential datasets suitable for Deep Learning models.
- Develop SimpleRNN, LSTM, and GRU models.
- Optimize model parameters through hyperparameter tuning.
- Evaluate and compare model performance.
- Deploy the best-performing model using Streamlit.

The final system should be capable of forecasting future Tesla stock prices for multiple prediction horizons.

---

3. Business Use Cases

Stock price prediction has several practical applications in the financial industry.

Automated trading systems can use model predictions to generate buy and sell signals automatically. Investment firms can use forecasts to optimize portfolio allocations and manage financial risk. Long-term investors can use future price trends to determine whether to hold, buy, or sell assets.

The model can also support company valuation, competitor analysis, financial forecasting, and research into advanced time-series forecasting techniques. Furthermore, the project provides a foundation for incorporating additional features such as sentiment analysis, macroeconomic indicators, and alternative financial data.

---

4. Dataset Description

The dataset used for this project contains historical Tesla stock market information. Each record represents a single trading day and includes multiple market indicators.

The dataset contains the following attributes:

- Date
- Open
- High
- Low
- Close
- Adj Close
- Volume

Among these features, the Closing Price was selected as the target variable because it represents the final trading price of Tesla stock for each trading day and is commonly used in financial forecasting.

	A	B	C	D	E	F	G
1	Date	Open	High	Low	Close	Adj Close	Volume
2	29-06-2010	19	25	17.540001	23.889999	23.889999	18766300
3	30-06-2010	25.790001	30.42	23.299999	23.83	23.83	17187100
4	01-07-2010	25	25.92	20.27	21.959999	21.959999	8218800
5	02-07-2010	23	23.1	18.709999	19.200001	19.200001	5139800
6	06-07-2010	20	20	15.83	16.110001	16.110001	6866900
7	07-07-2010	16.4	16.629999	14.98	15.8	15.8	6921700
8	08-07-2010	16.139999	17.52	15.57	17.459999	17.459999	7711400
9	09-07-2010	17.58	17.9	16.549999	17.4	17.4	4050600
10	12-07-2010	17.950001	18.07	17	17.049999	17.049999	2202500
11	13-07-2010	17.389999	18.639999	16.9	18.139999	18.139999	2680100
12	14-07-2010	17.940001	20.15	17.76	19.84	19.84	4195200
13	15-07-2010	19.940001	21.5	19	19.889999	19.889999	3739800
14	16-07-2010	20.700001	21.299999	20.049999	20.639999	20.639999	2621300
15	19-07-2010	21.370001	22.25	20.92	21.91	21.91	2486500
16	20-07-2010	21.85	21.85	20.049999	20.299999	20.299999	1825300
17	21-07-2010	20.66	20.9	19.5	20.219999	20.219999	1252500
18	22-07-2010	20.5	21.25	20.370001	21	21	957800
19	23-07-2010	21.190001	21.559999	21.059999	21.290001	21.290001	653600
20	26-07-2010	21.5	21.5	20.299999	20.950001	20.950001	922200
21	27-07-2010	20.91	21.18	20.26	20.549999	20.549999	619700
22	28-07-2010	20.549999	20.9	20.51	20.719999	20.719999	467200
23	29-07-2010	20.77	20.879999	20	20.35	20.35	616000
24	30-07-2010	20.200001	20.440001	19.549999	19.940001	19.940001	426900
25	02-08-2010	20.5	20.969999	20.33	20.92	20.92	718100
26	03-08-2010	21	21.950001	20.82	21.950001	21.950001	1230500

5. Data Cleaning and Preprocessing

Data quality is critical for building accurate predictive models. Therefore, several preprocessing steps were performed before model development.

The dataset was first loaded using Pandas and examined for missing values, duplicate records, and inconsistent data types. The Date column was converted into datetime format and used for chronological ordering of records.

No significant missing values were observed in the dataset. Duplicate entries were checked and removed if present. Since stock price prediction is a time-series problem, maintaining temporal order was essential during preprocessing.

To improve model convergence and training stability, MinMaxScaler was applied to normalize stock prices into the range of 0 to 1. Normalization ensures that large stock price values do not dominate the learning process.

Duplicate Values

```
# Dataset Duplicate Value Count
df.describe()
```

[88]

	Open	High	Low	Close	Adj Close	Volume
count	2416.000000	2416.000000	2416.000000	2416.000000	2416.000000	2.416000e+03
mean	186.271147	189.578224	182.916639	186.403651	186.403651	5.572722e+06
std	118.740163	120.892329	116.857591	119.136020	119.136020	4.987809e+06
min	16.139999	16.629999	14.980000	15.800000	15.800000	1.185000e+05
25%	34.342498	34.897501	33.587501	34.400002	34.400002	1.899275e+06
50%	213.035004	216.745002	208.870002	212.960007	212.960007	4.578400e+06
75%	266.450012	270.927513	262.102501	266.774994	266.774994	7.361150e+06
max	673.690002	786.140015	673.520020	780.000000	780.000000	4.706500e+07

Missing Values/Null Values

```
# Missing Values/Null Values Count
df.isnull().sum()
```

[89]

```
Date         0
Open         0
High         0
Low          0
Close        0
Adj Close    0
Volume       0
dtype: int64
```

```
# Visualizing the missing values
df.duplicated().sum()
```

[90]

```
np.int64(0)
```

1. Handling Missing Values

Generate | Code | Markdown

```
df.isnull().sum()
```

[110] Python

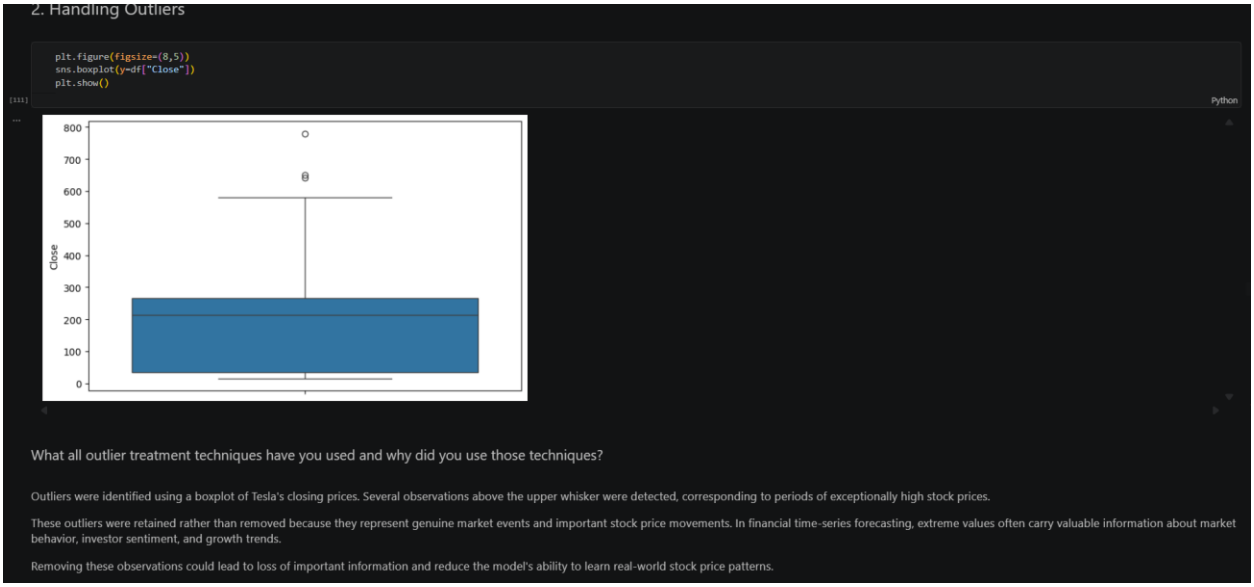
```
Open            0
High            0
Low             0
Close           0
Adj Close       0
Volume          0
MA30           29
MA100          99
Daily_Return    1
dtype: int64
```

What all missing value imputation techniques have you used and why did you use those techniques?

Missing values were identified only in the engineered features MA30, MA100, and Daily_Return. These missing values were not caused by data quality issues but were generated naturally during feature engineering.

• Daily_Return contains 1 missing value because the first observation has no previous day available for return calculation. • MA30 contains 29 missing values because a 30-day moving average requires the previous 30 observations. • MA100 contains 99 missing values because a 100-day moving average requires the previous 100 observations.

Instead of applying mean or median imputation, the rows containing these initial missing values were removed using dropna(). This approach preserves the integrity of the time-series data and prevents introducing artificial values that could negatively affect model training.

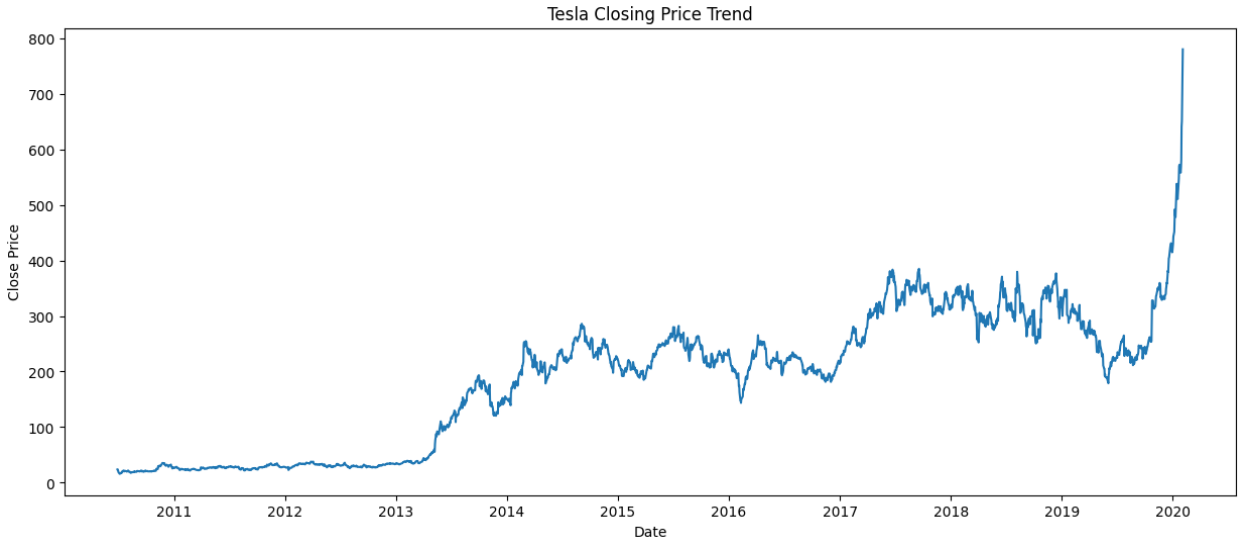


---

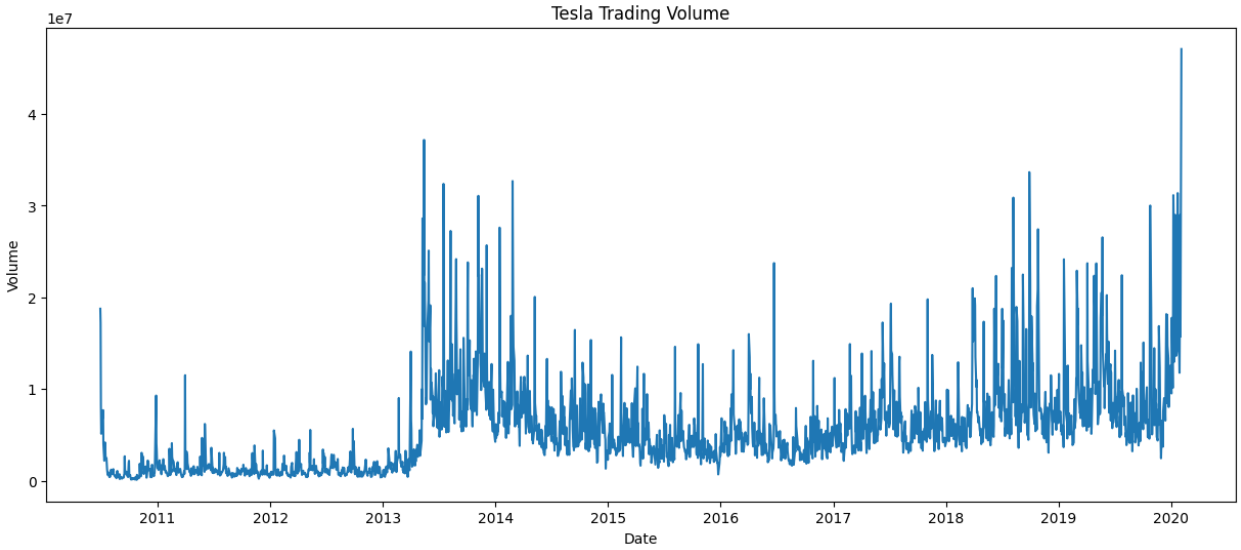
6. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was conducted to understand Tesla stock behavior over time and identify useful trends for modeling.

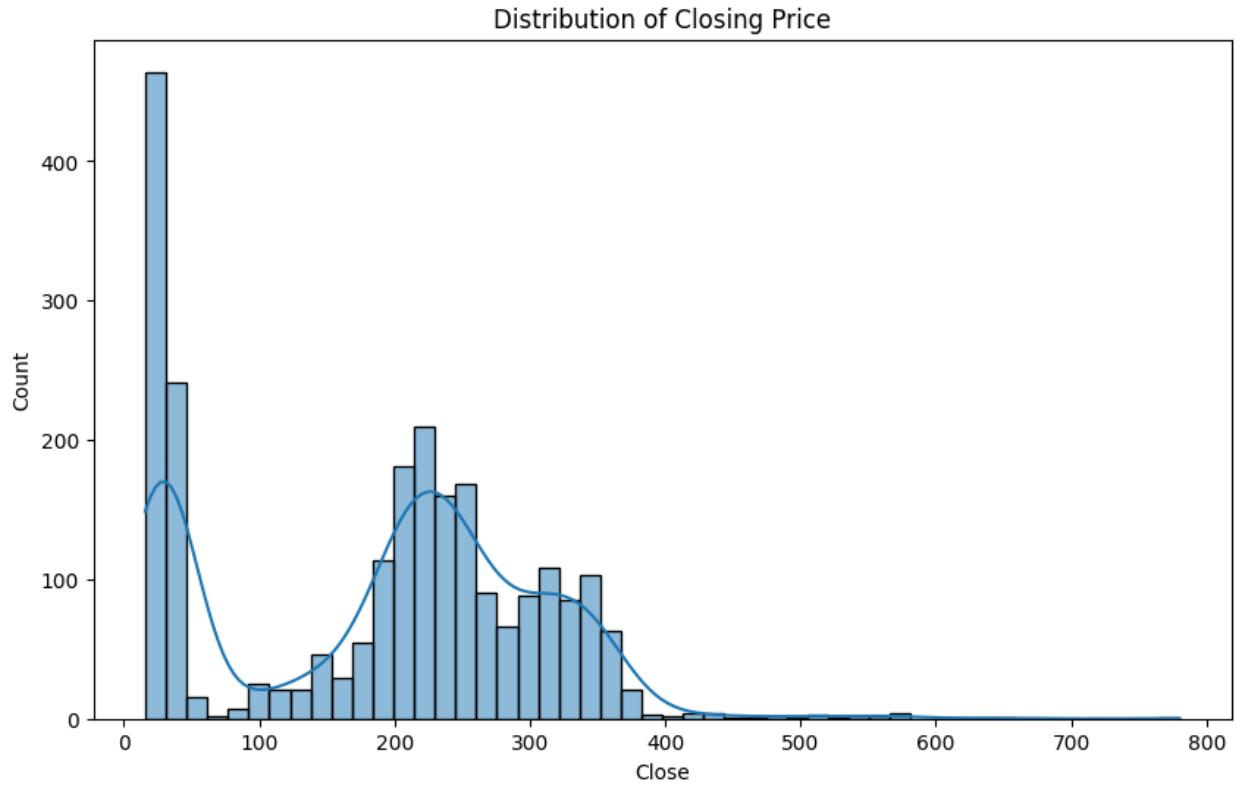
The historical closing price trend revealed a significant long-term growth pattern accompanied by periods of high volatility. Several sharp upward and downward movements were observed, indicating strong market fluctuations.



Trading volume analysis showed periods of unusually high investor activity corresponding to major market events and company announcements.

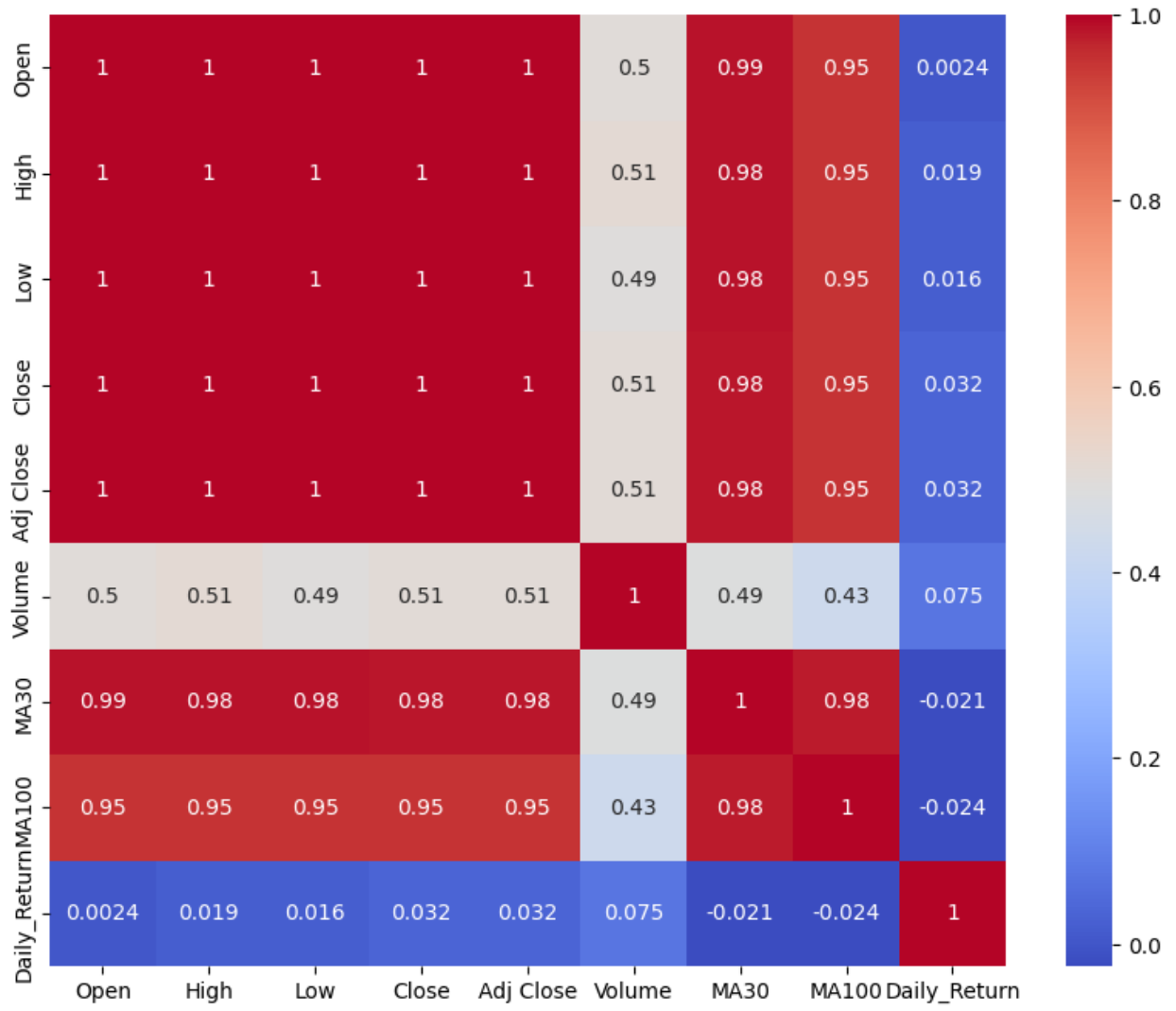


Distribution plots indicated that stock prices were positively skewed and contained several high-value observations.



Moving Average analysis was performed using 30-day and 100-day windows. Moving averages helped smooth short-term fluctuations and identify long-term trends.

Correlation analysis showed strong relationships between Open, High, Low, and Close prices.



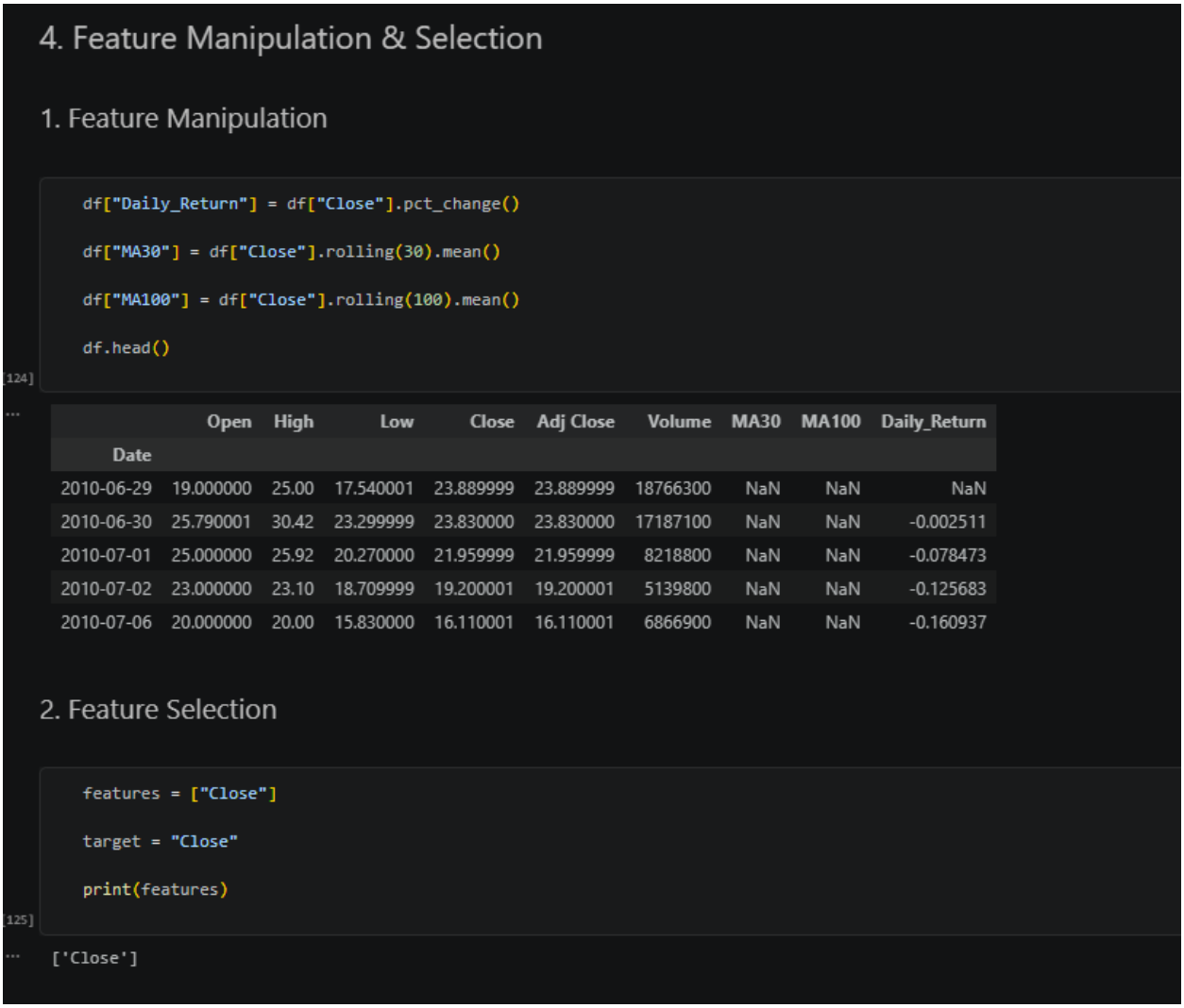
7. Feature Engineering

Feature engineering was performed to enhance predictive capability.

A Daily Return feature was created to measure percentage change in stock prices. Moving averages (MA30 and MA100) were also calculated to capture short-term and long-term trends.

For sequence generation, a sliding window approach was used. A sequence length of 60 trading days was selected, meaning that the previous 60 days were used to predict the next day's closing price.

This transformed the original tabular dataset into a supervised learning format suitable for recurrent neural networks.



8. Deep Learning Model Development

Three Deep Learning architectures were developed and evaluated.

8.1 SimpleRNN Model

The SimpleRNN model was implemented as the baseline architecture. It consisted of a SimpleRNN layer, dropout regularization, and a dense output layer.

Although SimpleRNN is capable of learning sequential patterns, it struggles to capture long-term dependencies due to the vanishing gradient problem.

ML Model - 1 RNN

```
# ========================================
# Time Series Data Preparation for RNN/LSTM/GRU
# ========================================

import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Remove NaN values generated by feature engineering
df_model = df.dropna().copy()

# Use Closing Price for prediction
data = df_model[['Close']]

# Scale data
scaler = MinMaxScaler(feature_range=(0,1))
scaled_data = scaler.fit_transform(data)

# Create sequences
sequence_length = 60

X = []
y = []

for i in range(sequence_length, len(scaled_data)):
    X.append(scaled_data[i-sequence_length:i, 0])
    y.append(scaled_data[i, 0])

X = np.array(X)
y = np.array(y)

# Train-Test Split (80:20)
train_size = int(len(X) * 0.8)

X_train = X[:train_size]
X_test = X[train_size:]

y_train = y[:train_size]
y_test = y[train_size:]

# Reshape for RNN/LSTM/GRU
X_train = X_train.reshape(
    X_train.shape[0],
    X_train.shape[1],
    1
)

X_test = X_test.reshape(
    X_test.shape[0],
    X_test.shape[1],
    1
)

print("="*50)
print("Training Data Shape")
print(X_train.shape)

print("\nTesting Data Shape")
print(X_test.shape)

print("\nTarget Train Shape")
print(y_train.shape)

print("\nTarget Test Shape")
print(y_test.shape)

print("="*50)
```

```
==================================================
Training Data Shape
(1805, 60, 1)

Testing Data Shape
(452, 60, 1)

Target Train Shape
(1805,)

Target Test Shape
(452,)
==================================================
```

8.2 LSTM Model

The LSTM model was developed to overcome the limitations of SimpleRNN. LSTM uses memory cells and gating mechanisms to retain important information over longer periods.

The architecture included an LSTM layer, dropout layer, and dense output layer.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

lstm_model = Sequential([
    LSTM(
        units=50,
        return_sequences=False,
        input_shape=(X_train.shape[1], X_train.shape[2])
    ),

    Dropout(0.2),

    Dense(25, activation='relu'),

    Dense(1)
])

lstm_model.compile(
    optimizer='adam',
    loss='mse'
)

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

history_lstm = lstm_model.fit(
    X_train,
    y_train,
    validation_split=0.1,
    epochs=50,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

lstm_pred = lstm_model.predict(X_test)

print("Prediction Shape:", lstm_pred.shape)
```

8.3 GRU Model

GRU is a simplified version of LSTM that combines memory retention and computational efficiency. The model consists of update and reset gates, allowing it to learn temporal dependencies effectively while requiring fewer parameters.

The GRU model architecture included a GRU layer, dropout layer, and dense output layer.

9. Hyperparameter Optimization

To improve model performance, Random Search Hyperparameter Tuning was employed.

The following parameters were optimized:

- Number of hidden units
- Dropout rate

Random Search was selected because it explores a diverse set of parameter combinations while remaining computationally efficient.

The best GRU configuration consisted of 32 units and a dropout rate of 0.1.

```
Trial 5 Complete [00h 00m 07s]
val_loss: 0.00018737860955297947

Best val_loss So Far: 0.00018289497529622167
Total elapsed time: 00h 00m 48s
{'units': 32, 'dropout': 0.1}
```

```
Epoch 1/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0135 - val_loss: 0.0063
Epoch 2/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 9.3452e-04 - val_loss: 2.9217e-04
Epoch 3/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 5.1370e-04 - val_loss: 2.7350e-04
Epoch 4/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 4.5455e-04 - val_loss: 3.3371e-04
Epoch 5/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 4.1967e-04 - val_loss: 2.4367e-04
Epoch 6/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 3.7186e-04 - val_loss: 2.3698e-04
Epoch 7/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 3.4985e-04 - val_loss: 2.8124e-04
Epoch 8/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 3.1887e-04 - val_loss: 3.0551e-04
Epoch 9/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 3.4869e-04 - val_loss: 2.1418e-04
Epoch 10/50
51/51 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 3.0025e-04 - val_loss: 2.1365e-04
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
MAE : 0.06651346033132977
RMSE: 0.08073053224051352
R2 : 0.31673387145257803
```

---

10. Model Evaluation

Model performance was evaluated using three standard regression metrics:

Mean Absolute Error (MAE)

Root Mean Squared Error (RMSE)

R^2 Score

MAE measures the average prediction error. RMSE penalizes larger errors more heavily. R^2 Score indicates how well the model explains variance in the data.

The evaluation results are presented below.

SimpleRNN: MAE = 11.94 RMSE = 18.30 R^2 = 0.9389

LSTM: MAE = 0.0130 RMSE = 0.0198 R^2 = 0.9591

GRU: MAE = 0.0105 RMSE = 0.0173 R^2 = 0.9686

Metric	Value
MAE	11.94
RMSE	18.30
R^2 Score	0.9389
Validation Loss	0.000208

Metric	Original LSTM	Tuned LSTM
MAE	0.0130	0.0250
RMSE	0.0198	0.0328
R^2 Score	0.9591	0.8875
Validation Loss	0.000195	0.000335

```
MAE : 0.010523312264425261
RMSE: 0.017313692951576292
R2 : 0.9685736688003762
```

---

11. Model Comparison and Selection

A comparative analysis was performed to identify the most effective forecasting model.

The GRU model achieved the lowest prediction error and highest R^2 Score among all architectures. It also demonstrated stable training behavior and efficient computation.

Consequently, GRU was selected as the final deployment model.

---

12. Streamlit Deployment

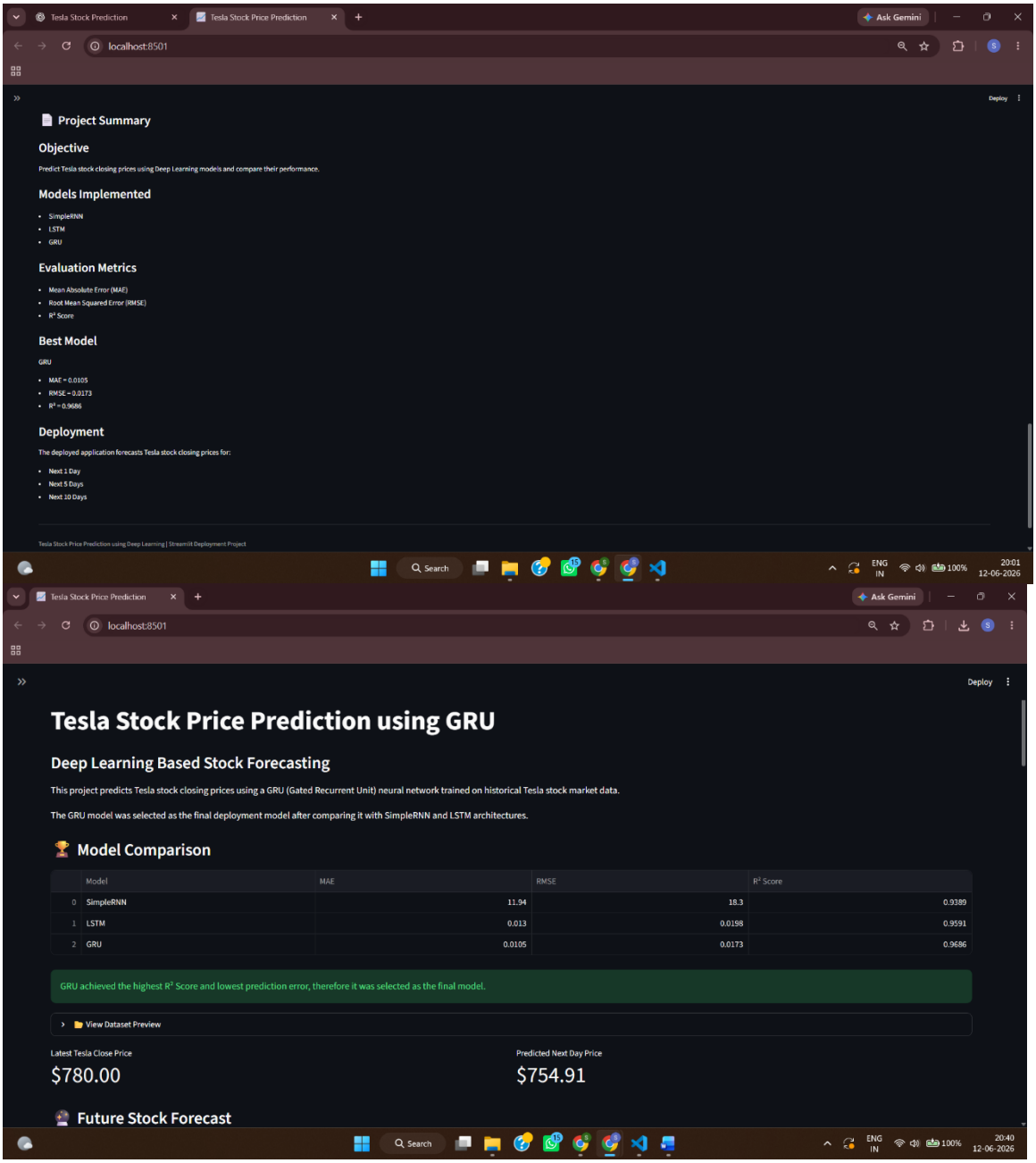
The best-performing GRU model was deployed using Streamlit.

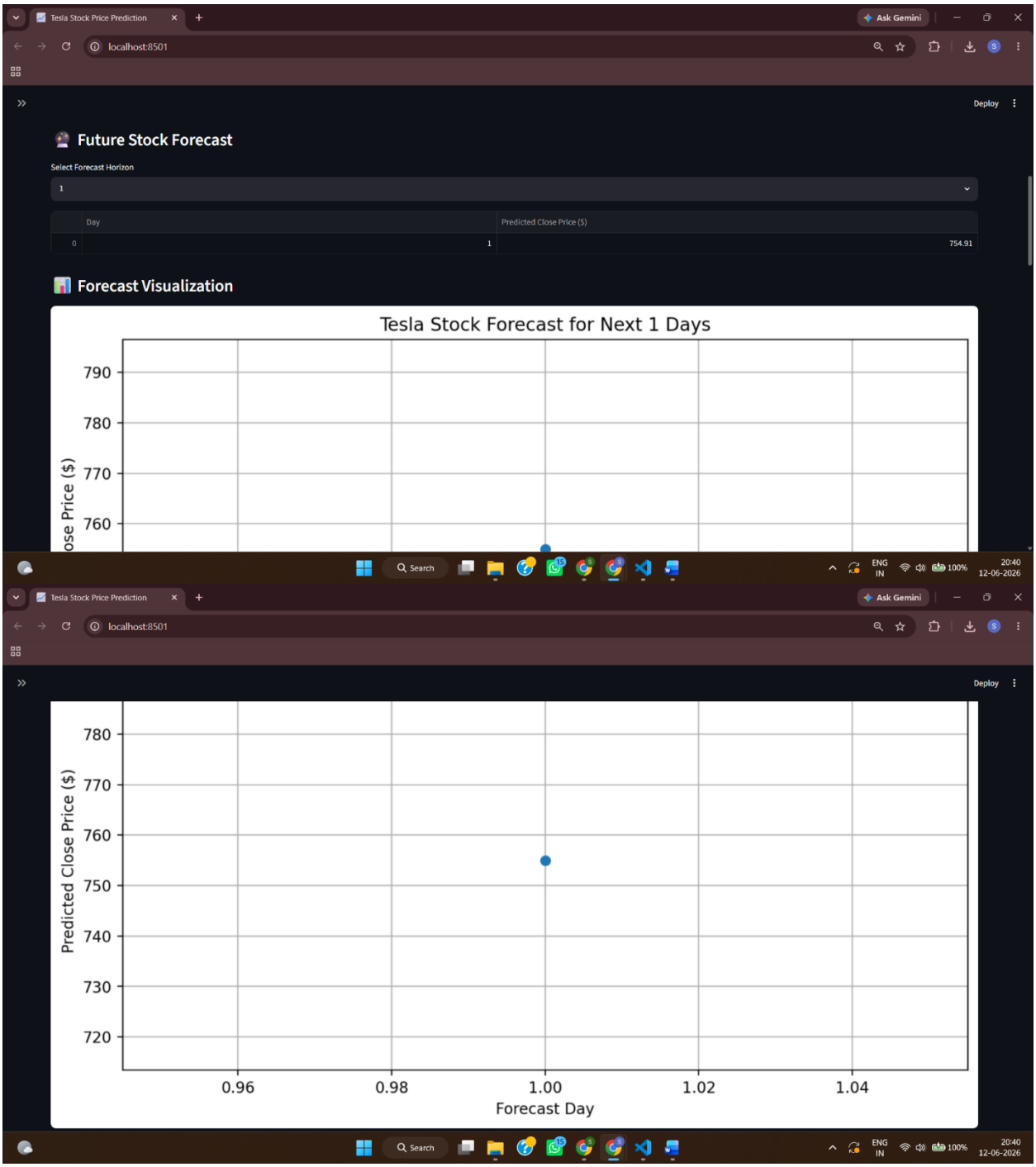
The deployed application allows users to:

- View Tesla historical stock data
- Compare Deep Learning models

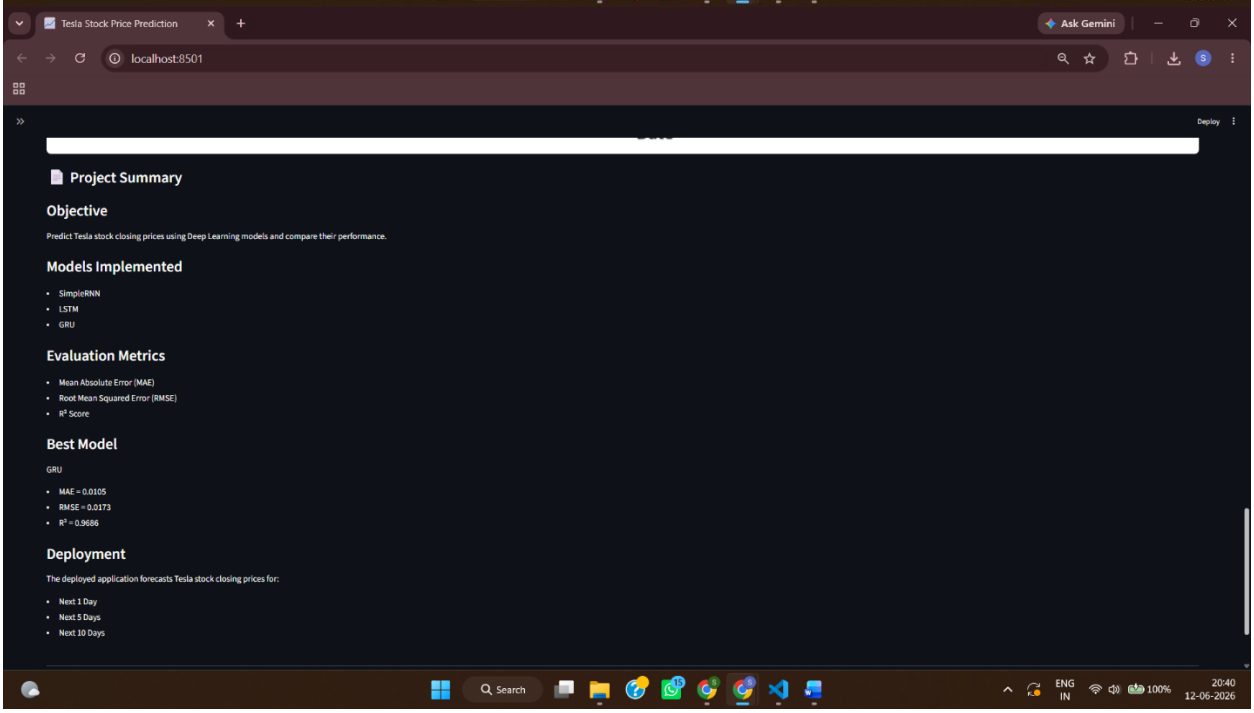
- Predict the next trading day price
- Forecast the next 5 days
- Forecast the next 10 days
- Visualize future trends

The deployment demonstrates the practical applicability of Deep Learning in financial forecasting.




Tesla Stock Price Prediction
Ask Gemini
localhost:8501
Deploy
Future Stock Forecast
Select Forecast Horizon
1
Day
Predicted Close Price ($)
0
1
754.91
Forecast Visualization
Tesla Stock Forecast for Next 1 Days
790
780
770
760
ose Price ($)
Search
ENG
IN
100%
20:40
12-06-2026
Tesla Stock Price Prediction
Ask Gemini
localhost:8501
Deploy
780
770
760
750
740
730
720
Predicted Close Price ($)
0.96
0.98
1.00
1.02
1.04
Forecast Day
Search
ENG
IN
100%
20:40
12-06-2026


Tesla Stock Price Prediction
Ask Gemini
localhost:8501
Deploy
Historical Tesla Closing Price Trend
Closing Price ($)
800
700
600
500
400
300
200
100
0
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
Date
Search
ENG
IN
100%
20:40
12-06-2026


Tesla Stock Price Prediction
Ask Gemini
localhost:8501
Deploy
Project Summary
Objective
Predict Tesla stock closing prices using Deep Learning models and compare their performance.
Models Implemented
SimpleRNN
LSTM
GRU
Evaluation Metrics
Mean Absolute Error (MAE)
Root Mean Squared Error (RMSE)
R² Score
Best Model
GRU
MAE = 0.0105
RMSE = 0.0173
R² = 0.9686
Deployment
The deployed application forecasts Tesla stock closing prices for:
Next 1 Day
Next 5 Days
Next 10 Days
Search
ENG
IN
100%
20:40
12-06-2026

13. Conclusion

This project successfully developed and compared three Deep Learning architectures for Tesla stock price forecasting. Extensive data preprocessing, exploratory analysis, feature engineering, and hyperparameter tuning were performed to improve predictive performance.

Among all models, GRU emerged as the best-performing architecture with an R^2 Score of 0.9686, demonstrating its ability to capture temporal dependencies effectively while maintaining computational efficiency.

The deployed Streamlit application provides an interactive forecasting platform capable of generating future stock price predictions. The results indicate that Deep Learning models can serve as valuable tools for stock market analysis and decision support.

---

14. Future Work

Future improvements may include integrating real-time stock market APIs, news sentiment analysis, social media signals, macroeconomic indicators, and Transformer-based forecasting models. Expanding the framework to support multiple stocks and portfolio-level forecasting can further enhance its practical value.